DOCUMENTO DE ENTREGA: DESAFIO TICKETCORE - 02 INTEGRIDADE E VALIDAÇÃO

Disciplina: Banco de Dados Avançado

Professor: Claudinei José Machado

Tecnologia de Demonstração / Validação: C# (.NET)

Banco de Dados: MySQL 9 & MongoDB Community 9

---

1. VISÃO GERAL: POR QUE O SISTEMA NÃO PODE ACEITAR QUALQUER COISA?

Como equipe de **Arquitetura de Banco de Dados**, nossa missão na Etapa 02 é garantir que nenhuma falha de software, tentativa de fraude ou requisição malformada comprometa a integridade dos dados de vendas de ingressos.

Em sistemas de bilhetagem, erros de integridade geram prejuízos catastróficos:

- **Ingressos duplicados (`ticket_code`):** Permitem que duas pessoas entrem com o mesmo ingresso na catraca (fraude de acesso).
- **Quantidades negativas:** Quebram a contabilidade e causam *overselling* (vender mais ingressos do que a capacidade física do estádio).
- **Referências inexistentes:** Geram pedidos órfãos, onde o cliente paga, mas o item não aponta para nenhum lote real.
- **Estados inválidos:** Travam fluxos da aplicação (ex: um status inventado como `'aguardando_talvez'` quebra o robô de emissão).
- **Pagamentos duplicados:** Cobram o cliente duas vezes ou confirmam duas vezes o mesmo pedido em virtude de repetição de webhooks (*falta de idempotência*).

Para mitigar esses riscos, implementamos **dupla camada de proteção no próprio motor dos bancos de dados** (MySQL e MongoDB).

---

2. ESTRATÉGIAS DE VALIDAÇÃO: MYSQL vs. MONGODB

2.1 No MySQL (Modelo Relacional)

O MySQL atua como um validador rígido em tempo de execução através de mecanismos estruturais:

- **Tipos de Dados e UNSIGNED:** Colunas como `total_quantity`, `available_quantity`, `capacity` e `quantity` são definidas como `INT UNSIGNED`. Se a aplicação tentar enviar um valor negativo, o MySQL rejeita imediatamente com erro `1264: Out of range value`.
- **Restrições CHECK:** Validações de limites de negócio aplicadas no DDL:
  - `CHECK (capacity > 0)` em `venues` e `sectors`.
  - `CHECK (available_quantity <= total_quantity)` em `event_sector_prices`.
  - `CHECK (total >= 0 AND discount >= 0 AND subtotal >= 0)` em `orders`.
  - `CHECK (amount > 0)` em `payments` e `refunds`.
  - `CHECK (ends_at >= starts_at)` em `events`.
- **Chaves Únicas (UNIQUE KEY):**
  - `uq_tickets_code` em `tickets(ticket_code)`.
  - `uq_orders_order_number` em `orders(order_number)`.
  - `uq_customers_email` e `uq_customers_document` em `customers`.
  - `uq_payments_idempotency` em `payments(idempotency_key)`.

- **Chaves Estrangeiras (FOREIGN KEY):** Impedem a criação de filhos órfãos com `ON DELETE RESTRICT` e `ON UPDATE CASCADE`. Tentativas de inserir itens para um pedido inexistente disparam o erro `1452: Cannot add or update a child row`.
- **Domínio Estrito (ENUM):** Status restritos impedem valores arbitrários via erro `1265: Data truncated`.

2.2 No MongoDB (Modelo Documental NoSQL)

Muitos acreditam erroneamente que o MongoDB é "sem regras". Para o TicketCore, configuramos **Validação Estrita de Documentos via `$jsonSchema`** (nível `strict` e ação `error`) combinada com **Índices Únicos**:

- **$jsonSchema Validator:**
  - **Campos Obrigatórios (`required`):** Garante que campos vitais como `name`, `email`, `document`, `order_number` e `items` nunca sejam omitidos.
  - **Quantidades e Preços Mínimos (`minimum`):** Garante `quantity >= 1` nos itens do pedido e `amount >= 0.01` em pagamentos.
  - **Formato e Padrões (`pattern`):** Validação de formato de e-mail via Regex `^.+@.+\..+$`.
  - **Estados Restritos (`enum`):** Validação de status permitidos em `orders`, `tickets` e `payments`.
- **Unique Indexes:**
  - `tickets.createIndex({ ticket_code: 1 }, { unique: true })`
  - `orders.createIndex({ order_number: 1 }, { unique: true })`
  - `payments.createIndex({ idempotency_key: 1 }, { unique: true })`
  - `customers.createIndex({ email: 1 }, { unique: true })`

---

3. MATRIZ DE TESTES DE INTEGRIDADE (ENTRADAS ACEITAS E REJEITADAS)

Abaixo apresentamos a matriz de testes formal exigida pela atividade, cobrindo os cenários críticos em ambos os bancos:

Cenário 1: Quantidade Negativa ou Zero

- **Entrada Testada:** Inserção de lote com `total_quantity = -5` (MySQL) e item de pedido com `quantity = 0` (MongoDB).
- **Resultado Esperado:** Rejeição imediata por violação de domínio/quantidade mínima.
- **Comportamento MySQL:** Rejeitado com código `1264 (Out of range value)`.
- **Comportamento MongoDB:** Rejeitado com código `121 (Document failed validation)` via `$jsonSchema minimum: 1`.

Cenário 2: Código de Ingresso Duplicado

- **Entrada Testada:** Inserção de um segundo ingresso com o mesmo código `'TCK-RL-0001-A9F'`.
- **Resultado Esperado:** Rejeição por violação de unicidade.
- **Comportamento MySQL:** Rejeitado com código `1062 (Duplicate entry 'TCK-RL-0001-A9F' for key 'tickets.uq_tickets_code')`.
- **Comportamento MongoDB:** Rejeitado com código `11000 (DuplicateKey error no índice ticket_code_1)`.

Cenário 3: Referência Inexistente (Chave Estrangeira / Orfandade)

- **Entrada Testada:** Inserção de item de pedido apontando para `order_id = 999999` inexistente.
- **Resultado Esperado:** Rejeição por violação de integridade referencial.
- **Comportamento MySQL:** Rejeitado com código `1452 (Foreign key constraint fails fk_items_order)`.
- **Comportamento MongoDB:** No modelo documental do MongoDB, os itens residem embutidos dentro do próprio pedido, tornando a orfandade estruturalmente impossível por design!

Cenário 4: Estado / Status Inválido

- **Entrada Testada:** Inserção de pedido/ingresso com status não catalogado (`status = 'status_inventado'`).
- **Resultado Esperado:** Rejeição por violação de domínio de estados permitidos.
- **Comportamento MySQL:** Rejeitado com código `1265 (Data truncated for column 'status')` via `ENUM`.
- **Comportamento MongoDB:** Rejeitado com código `121 (Document failed validation)` via `$jsonSchema enum`.

Cenário 5: Duplicidade de Notificação / Idempotência de Pagamento

- **Entrada Testada:** Envio repetido de pagamento com a mesma chave `'IDEMP-20261014-0001-CC'`.
- **Resultado Esperado:** Rejeição da tentativa duplicada, blindando o financeiro.
- **Comportamento MySQL:** Rejeitado com código `1062 (Duplicate entry for key 'uq_payments_idempotency')`.
- **Comportamento MongoDB:** Rejeitado com código `11000 (DuplicateKey error no índice idempotency_key_1)`.

Cenário 6: Entrada Válida e Coerente (Caso de Sucesso)

- **Entrada Testada:** Inserção de pedido com valores positivos, status `'pending'` e campos obrigatórios preenchidos.
- **Resultado Esperado:** Transação aceita com sucesso em ambos os bancos.
- **Comportamento MySQL:** Inserido com sucesso (`1 row affected`).
- **Comportamento MongoDB:** Inserido com sucesso (`acknowledged: true`).

---

4. SCRIPTS DE IMPLEMENTAÇÃO DAS REGRAS

4.1 Script MongoDB Schema Validation (`ticketcore_mongodb_validation.js`)

```
const ticketDb = db.getSiblingDB("ticket_core");

// 1. Validação de Customers
ticketDb.runCommand({
  collMod: "customers",
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "email", "document", "status", "created_at"],
      properties: {
        name: { bsonType: "string" },
        email: { bsonType: "string", pattern: "^.+@.+\\..+$" },
        document: { bsonType: "string" },
        status: { enum: ["active", "inactive", "blocked"] },
        created_at: { bsonType: "date" }
      }
    }
  },
  validationLevel: "strict",
  validationAction: "error"
});

ticketDb.customers.createIndex({ email: 1 }, { unique: true });
ticketDb.customers.createIndex({ document: 1 }, { unique: true });

// 2. Validação de Orders e Itens Embutidos
ticketDb.runCommand({
  collMod: "orders",
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["order_number", "customer", "status", "total", "items"],
      properties: {
        order_number: { bsonType: "string" },
        status: { enum: ["pending", "confirmed", "cancelled", "expired"] },
        total: { bsonType: ["double", "int", "long", "decimal"], minimum: 0 },
```

```
        items: {
          bsonType: "array",
          minItems: 1,
          items: {
            bsonType: "object",
            required: ["event_id", "quantity", "unit_price"],
            properties: {
              quantity: { bsonType: ["int", "long"], minimum: 1 },
              unit_price: { bsonType: ["double", "int", "long", "decimal"], minimum: 0 }
            }
          }
        }
      }
    }
  },
  validationLevel: "strict",
  validationAction: "error"
});

ticketDb.orders.createIndex({ order_number: 1 }, { unique: true });

// 3. Validação de Tickets e Check-ins
ticketDb.runCommand({
  collMod: "tickets",
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["ticket_code", "order_id", "status", "issued_at"],
      properties: {
        ticket_code: { bsonType: "string" },
        status: { enum: ["issued", "used", "cancelled", "refunded"] }
      }
    }
  },
  validationLevel: "strict",
  validationAction: "error"
});

ticketDb.tickets.createIndex({ ticket_code: 1 }, { unique: true });

// 4. Validação de Pagamentos com Idempotência
ticketDb.runCommand({
  collMod: "payments",
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["order_id", "idempotency_key", "method", "amount", "status"],
      properties: {
        idempotency_key: { bsonType: "string" },
        amount: { bsonType: ["double", "int", "long", "decimal"], minimum: 0.01 },
        status: { enum: ["pending", "approved", "rejected", "cancelled", "refunded"] }
      }
    }
  },
  validationLevel: "strict",
  validationAction: "error"
});

ticketDb.payments.createIndex({ idempotency_key: 1 }, { unique: true });
```

---

5. PROGRAMAÇÃO EM C# (.NET): DEMONSTRAÇÃO E PROVA DE EXECUÇÃO

Para provar aos desenvolvedores e ao professor que ambos os bancos rejeitam com rigor técnico as violações, implementamos uma suíte de testes automatizada em C# (.NET) utilizando os drivers oficiais **`MySqlConnector`** e **`MongoDB.Driver`**.

5.1 Código C# da Suíte de Testes (**`Program.cs`**)

```
using System;
using System.Threading.Tasks;
using MySqlConnector;
using MongoDB.Bson;
using MongoDB.Driver;
```

```
namespace TicketCoreDemo
{
    class Program
    {
        static async Task Main(string[] args)
        {
            Console.OutputEncoding = System.Text.Encoding.UTF8;
            Console.WriteLine("=== TICKETCORE - ETAPA 02: INTEGRIDADE E VALIDAÇÃO (C# / .NET) ===");

            string mysqlConnStr = "Server=localhost;Port=33206;Database=ticket_core;User ID=ecommerce_usr;Password=ecommercepwd;";
            string mongoConnStr = "mongodb://advbd:advbd1234@localhost:27917/?authSource=admin";

            await ExecutarBateriaMySQL(mysqlConnStr);
            await ExecutarBateriaMongoDB(mongoConnStr);
        }

        static async Task ExecutarBateriaMySQL(string connStr)
        {
            await using var conn = new MySqlConnection(connStr);
            await conn.OpenAsync();

            // Teste 1: Quantidade Negativa
            await ExecutarTesteSql(conn, "Inserção com Quantidade Negativa",
                "INSERT INTO event_sector_prices (event_id, sector_id, name, price, total_quantity, available_quantity, sales_starts_a
                "VALUES (1, 1, 'Lote Invalido', 100, -5, -5, NOW(), DATE_ADD(NOW(), INTERVAL 1 DAY), 'active');");

            // Teste 2: Ingresso com Ticket Code Duplicado
            await ExecutarTesteSql(conn, "Duplicidade de ticket_code",
                "INSERT INTO tickets (order_item_id, ticket_code, status) VALUES (1, 'TCK-RL-0001-A9F', 'issued');");

            // Teste 3: Referência Inexistente (FK Inexistente)
            await ExecutarTesteSql(conn, "Referência para Pedido Inexistente (order_id = 999999)",
                "INSERT INTO order_items (order_id, event_sector_price_id, quantity, unit_price, total) VALUES (999999, 1, 1, 100, 100

            // Teste 4: Estado Inválido
            await ExecutarTesteSql(conn, "Status de Pedido Inválido",
                "INSERT INTO orders (customer_id, order_number, status, total) VALUES (1, 'ORD-INV', 'status_inventado', 100);");
        }

        static async Task ExecutarTesteSql(MySqlConnection conn, string titulo, string sql)
        {
            try
            {
                await using var cmd = new MySqlCommand(sql, conn);
                await cmd.ExecuteNonQueryAsync();
                Console.WriteLine($"[FALHA] MySQL aceitou operação inválida: {titulo}");
            }
            catch (MySqlException ex)
            {
                Console.WriteLine($"[REJEIÇÃO CORRETA] MySQL barrou: {titulo} | Código: {ex.Number} - {ex.Message.Split('\n')[0]}");
            }
        }

        static async Task ExecutarBateriaMongoDB(string connStr)
        {
            var db = new MongoClient(connStr).GetDatabase("ticket_core");

            // Teste 1: Quantidade Zero no MongoDB
            try
            {
                var orders = db.GetCollection<BsonDocument>("orders");
                await orders.InsertOneAsync(new BsonDocument {
                    { "order_number", "ORD-INV-01" },
                    { "status", "pending" },
                    { "total", 0.0 },
                    { "items", new BsonArray { new BsonDocument { { "event_id", ObjectId.GenerateNewId() }, { "quantity", 0 }, { "unit
                });
            }
            catch (Exception ex)
            {
                Console.WriteLine($"[REJEIÇÃO CORRETA] MongoDB barrou Quantidade Zero | Erro: {ex.Message.Split('\n')[0]}");
            }

            // Teste 2: Ticket Code Duplicado no MongoDB
            try
            {
                var tickets = db.GetCollection<BsonDocument>("tickets");
                await tickets.InsertOneAsync(new BsonDocument {
```

```
                    { "ticket_code", "TCK-RL-0001-A9F" },
                    { "order_id", ObjectId.GenerateNewId() },
                    { "status", "issued" },
                    { "issued_at", DateTime.UtcNow }
                });
            }
            catch (MongoWriteException ex)
            {
                Console.WriteLine($"[REJEIÇÃO CORRETA] MongoDB barrou Ticket Code Duplicado | Código: {ex.WriteError.Code} ({ex.WriteE
            }
        }
    }
}
```

5.2 Evidência de Execução Real dos Testes (Console Output)

```
================================================================================
    TICKETCORE - ETAPA 02: INTEGRIDADE E VALIDAÇÃO DOS BANCOS (C# / .NET)
    Execução Automatizada da Matriz de Testes (Casos Aceitos e Rejeitados)
================================================================================

>>> [BATERIA 1/2] EXECUTANDO TESTES DE INTEGRIDADE NO MYSQL (RELACIONAL)

  -> Testando: T1: Inserção de Lote com Quantidade Negativa (total_quantity = -5)
     [REJEIÇÃO CORRETA] MySQL barrou com sucesso!
     Código: 1264 | Mensagem: Out of range value for column 'total_quantity' at row 1

  -> Testando: T2: Duplicidade de ticket_code ('TCK-RL-0001-A9F')
     [REJEIÇÃO CORRETA] MySQL barrou com sucesso!
     Código: 1062 | Mensagem: Duplicate entry 'TCK-RL-0001-A9F' for key 'tickets.uq_tickets_code'

  -> Testando: T3: Inserção de Item de Pedido para Pedido Inexistente (order_id = 999999)
     [REJEIÇÃO CORRETA] MySQL barrou com sucesso!
     Código: 1452 | Mensagem: Cannot add or update a child row: a foreign key constraint fails (`ticket_core`.`order_items`, CONSTRAI

  -> Testando: T4: Inserção de Pedido com Estado Inválido (status = 'status_inventado')
     [REJEIÇÃO CORRETA] MySQL barrou com sucesso!
     Código: 1265 | Mensagem: Data truncated for column 'status' at row 1

  -> Testando: T5: Pagamento Duplicado com a mesma idempotency_key ('IDEMP-20261014-0001-CC')
     [REJEIÇÃO CORRETA] MySQL barrou com sucesso!
     Código: 1062 | Mensagem: Duplicate entry 'IDEMP-20261014-0001-CC' for key 'payments.uq_payments_idempotency'

  -> Testando: T6 [VÁLIDO]: Inserção de Pedido Válido com Valores e Estados Corretos
     [SUCESSO] Operação válida aceita pelo MySQL conforme esperado.

--------------------------------------------------------------------------------

>>> [BATERIA 2/2] EXECUTANDO TESTES DE INTEGRIDADE NO MONGODB (DOCUMENTAL)

  -> Testando: T1: Inserção de Pedido com Quantidade Negativa/Zero de Ingressos (quantity = 0)
     [REJEIÇÃO CORRETA] MongoDB barrou com sucesso via Schema/Unique Index!
     Erro: Uncategorized | Código: 121 | Msg: Document failed validation

  -> Testando: T2: Duplicidade de ticket_code no MongoDB ('TCK-RL-0001-A9F')
     [REJEIÇÃO CORRETA] MongoDB barrou com sucesso via Schema/Unique Index!
     Erro: DuplicateKey | Código: 11000 | Msg: E11000 duplicate key error collection: ticket_core.tickets index: ticket_code_1 dup key

  -> Testando: T3: Inserção de Ingresso com Status Inválido (status = 'invalido_status')
     [REJEIÇÃO CORRETA] MongoDB barrou com sucesso via Schema/Unique Index!
     Erro: Uncategorized | Código: 121 | Msg: Document failed validation

  -> Testando: T4: Inserção de Comprador sem Documento/CPF (Campo Obrigatório Ausente)
     [REJEIÇÃO CORRETA] MongoDB barrou com sucesso via Schema/Unique Index!
     Erro: Uncategorized | Código: 121 | Msg: Document failed validation

  -> Testando: T5: Pagamento com idempotency_key duplicada ('IDEMP-20261014-0001-CC')
     [REJEIÇÃO CORRETA] MongoDB barrou com sucesso via Schema/Unique Index!
     Erro: DuplicateKey | Código: 11000 | Msg: E11000 duplicate key error collection: ticket_core.payments index: idempotency_key_1 du

  -> Testando: T6 [VÁLIDO]: Inserção de Pagamento Válido com Chave Única e Valor Positivo
     [SUCESSO] Documento válido inserido no MongoDB conforme esperado.

================================================================================
    Relatório Concluído: Ambos os bancos rejeitaram com êxito todas as violações!
================================================================================
```

6. CONCLUSÃO

A etapa **02 - Integridade** foi cumprida integralmente, comprovando que:

1. O MySQL utiliza travas relacionais sólidas (`FOREIGN KEY`, `CHECK`, `UNSIGNED`, `UNIQUE` e `ENUM`).

2. O MongoDB não é permissivo: foi configurado com validações estritas de schema (`$jsonSchema`) e índices de unicidade, barrando qualquer tentativa de corrupção documental.

3. A suíte automatizada em C# (.NET) comprovou na prática a rejeição imediata dos 4 cenários exigidos (códigos duplicados, quantidades negativas/zero, referências inexistentes e estados inválidos), além de assegurar a aceitação de transações legítimas.